

Design and Implementation of a Library Management System for Effective Book Cataloging, Borrowing and Returning Process

Student Name: [Your Full Name] **Matriculation Number:** [Your Matric Number] **Department:** [Department Name] **Faculty:** [Faculty Name] **Institution:** [University/College Name] **Supervisor:** [Supervisor Name] **Date:** August 2026

Table of Contents

- 1. Abstract
- 2. Introduction
- 3. Problem Statement
- 4. Objectives of the Study
- 5. Significance of the Study
- 6. Scope and Limitations
- 7. Literature Review
- 8. System Analysis
- 9. System Design
- 10. Implementation
- 11. System Features
- 12. Technologies Used
- 13. Database Design
- 14. System Architecture
- 15. Testing
- 16. Results and Discussion
- 17. Conclusion and Recommendations
- 18. References

1. Abstract

This project presents the design and implementation of a web-based Library Management System (LMS) aimed at improving the efficiency of book cataloging, borrowing, and returning processes in a library environment. The system was developed using React.js for the frontend, Node.js with Express for the backend, and MongoDB as the database. It provides separate interfaces for library members and library staff (librarians and administrators), each with role-based access control. Members can browse the book catalog, borrow available books, and track their borrowing history including due dates and overdue fines. Librarians can manage the full book catalog, register members, issue books, process returns, and calculate fines automatically. Administrators have full system access including the ability to create staff accounts. The system is fully deployed — the backend on Render and the frontend on Vercel — making it accessible from any device. The result is a modern, efficient, and user-friendly library management solution that eliminates the limitations of manual record-keeping.

2. Introduction

Libraries are fundamental institutions for knowledge acquisition and academic development. However, many libraries still rely on manual or semi-automated processes for managing books, members, and lending activities. These manual systems are prone to errors, inefficiencies, and difficulties in tracking records. As the volume of library transactions increases, the need for a reliable and automated management system becomes increasingly important.

A Library Management System (LMS) is software designed to automate the day-to-day operations of a library. It handles tasks such as cataloging books, registering members, tracking borrowed books, processing returns, and calculating overdue fines. A well-designed LMS reduces human error, saves time, and provides real-time information to both library staff and members.

This project presents a full-stack web-based Library Management System that was designed and implemented to address these challenges. The system provides a comprehensive digital platform that supports all core library operations while being accessible from any device through a modern web browser.

3. Problem Statement

Traditional library management in many institutions suffers from several challenges:

- **Manual record-keeping** leads to errors in tracking borrowed books and member details.
- **Lack of real-time information** makes it difficult for librarians to know which books are available at any given time.
- **Inefficient borrowing and returning processes** result in long queues and delays.
- **Difficulty in calculating overdue fines** due to the absence of automated date tracking.
- **Poor member experience** as members cannot check book availability or their borrowing history without physically visiting the library.
- **No clear separation of roles** between members and library staff, leading to unauthorized access to administrative functions.

These problems result in poor service delivery, loss of library resources, and reduced member satisfaction. There is a clear need for an automated, web-based solution that addresses all these issues comprehensively.

4. Objectives of the Study

The primary objectives of this project are:

1. To design and implement a web-based library management system that automates book cataloging, borrowing, and returning processes.
2. To develop a role-based access control system that differentiates between library members, librarians, and administrators.
3. To implement an automated fine calculation system for overdue books at a rate of \$0.50 per day.
4. To create a public member portal where library members can self-register, browse the catalog, and track their borrowing history.
5. To provide librarians with tools to manage books, members, and lending transactions efficiently.
6. To implement a secure authentication system using JSON Web Tokens (JWT).

7. To deploy the system to a cloud environment making it accessible from any device.
 8. To design a responsive user interface that works on both desktop and mobile devices.
-

5. Significance of the Study

This project is significant for the following reasons:

- **Automation of library operations** reduces manual workload and human error.
 - **Real-time availability tracking** ensures members and staff always know which books are available.
 - **Improved member experience** through a self-service portal for browsing and borrowing.
 - **Secure role-based access** protects sensitive administrative functions from unauthorized users.
 - **Cloud deployment** makes the system accessible anytime and anywhere without infrastructure costs.
 - **Automated email notifications** keep staff informed when new accounts are created.
 - **Scalability** — the system can be extended to support multiple library branches or larger book collections.
-

6. Scope and Limitations

Scope

The system covers the following functional areas:

- User registration and authentication for both members and staff
- Book catalog management with full CRUD operations
- Member management and registration
- Book borrowing and returning with date tracking
- Automatic overdue fine calculation
- Role-based dashboards for members, librarians, and administrators
- Email notifications for newly created staff accounts
- A first-run setup wizard for initial system configuration
- Full mobile responsiveness
- Cloud deployment on Render (backend) and Vercel (frontend)

Limitations

- The system does not support physical book barcode scanning.
 - Payment processing for fines is not included; fines are calculated but collection is manual.
 - Email notifications currently require manual Gmail App Password configuration.
 - The free tier of Render causes the backend server to sleep after 15 minutes of inactivity, resulting in a brief startup delay.
-

7. Literature Review

Several studies and existing systems have explored library management automation. Afolabi and Adeola (2019) emphasized that automated library systems significantly reduce the time required for book checkouts and returns compared to manual processes. Okafor et al. (2020) noted that web-based library systems improve accessibility and reduce the need for physical visits to check book availability.

Existing commercial systems such as Koha, OpenBiblio, and Evergreen provide comprehensive library management solutions. However, these systems are often complex to configure, require dedicated server infrastructure, and may be too feature-heavy for small to medium-sized libraries. This project addresses the need for a lightweight, modern, cloud-ready solution that maintains core functionality while being significantly easier to deploy and use.

The use of the MERN stack (MongoDB, Express, React, Node.js) for web application development has been widely validated in recent literature. Its component-based architecture, JavaScript consistency across frontend and backend, and strong community support make it an ideal choice for building scalable web applications (Subramanian, 2019).

Role-based access control (RBAC) is an established security model in information systems that restricts system access based on the roles of individual users. This project implements RBAC with three distinct roles: member, librarian, and administrator, each with clearly defined permissions.

8. System Analysis

8.1 Existing System Analysis

The existing manual system at most libraries involves physical registers for recording borrowed books, membership cards, and manual calculation of due dates and fines. This system has the following weaknesses:

- Records are easily lost or damaged
- No way to search for a book without physically scanning shelves
- Fine calculation is error-prone and inconsistent
- No way for members to check their borrowing history remotely
- Staff must manually cross-reference records to identify overdue books

8.2 Proposed System Analysis

The proposed web-based Library Management System addresses all identified weaknesses through:

- Digital storage of all records in a cloud-hosted MongoDB database
- Full-text search across title, author, and ISBN
- Automatic date tracking for borrowed books
- Automatic fine calculation at \$0.50 per day
- Member self-service portal for remote access
- Automated identification of overdue books through background status updates

8.3 Feasibility Study

Technical Feasibility: All technologies used (React, Node.js, MongoDB, JWT) are well-established, open-source, and freely available. Deployment platforms (Render, Vercel) offer free tiers suitable for the project scope.

Economic Feasibility: The system requires no paid software licenses. Hosting costs are zero on free tiers. The only potential cost is a paid Render plan if higher uptime is required.

Operational Feasibility: The system is designed with a clean, intuitive interface that requires minimal training for library staff and no technical knowledge from members.

9. System Design

9.1 System Architecture

The system follows a three-tier client-server architecture:



9.2 Use Case Diagram

The system supports the following primary use cases:

Library Member:

- Register account
- Log in / Log out
- Browse book catalog
- Search books by title, author, or category
- Borrow available books
- View borrowing history
- Check due dates and fines

Librarian:

- Log in / Log out

- Add, edit, delete books
- Register members
- Issue books to members
- Process book returns
- View borrowing records
- Monitor overdue books

Administrator:

- All librarian permissions
- Create librarian and admin accounts
- View system dashboard statistics

9.3 Database Entity Relationship

The system uses four main collections:

User (staff accounts)

- name, email, password (hashed), role (librarian/admin)

Member (library members)

- name, email, password (hashed), memberId (auto-generated), phone, address, membershipStatus, booksCurrentlyBorrowed, totalBorrowedAllTime

Book

- title, author, isbn, category, publisher, publishedYear, totalCopies, availableCopies, description, status (available/unavailable)

Borrow

- book (ref), member (ref), borrowDate, dueDate, returnDate, status (borrowed/returned/overdue), fine, finePaid

10. Implementation

10.1 Development Environment

- **Operating System:** Windows 10
- **Code Editor:** Visual Studio Code (Kiro IDE)
- **Version Control:** Git / GitHub
- **Node.js Version:** v24.14.1
- **Package Manager:** npm

10.2 Project Structure

```
library-project/  
├── server/                # Backend (Node.js + Express)
```

```

├── controllers/           # Business logic handlers
│   ├── authController.js
│   ├── bookController.js
│   ├── memberController.js
│   ├── borrowController.js
│   └── memberAuthController.js
├── models/               # MongoDB schemas
│   ├── User.js
│   ├── Member.js
│   ├── Book.js
│   └── Borrow.js
├── routes/               # API route definitions
│   ├── auth.js
│   ├── books.js
│   ├── members.js
│   ├── borrows.js
│   ├── memberAuth.js
│   └── memberPortal.js
├── middleware/           # Authentication middleware
│   ├── authMiddleware.js
│   └── memberAuthMiddleware.js
├── utils/
│   └── emailService.js    # Nodemailer email service
├── server.js             # Express app entry point
├── src/                  # Frontend (React + Vite)
│   ├── components/       # Librarian dashboard components
│   │   ├── Navbar.jsx
│   │   ├── Dashboard.jsx
│   │   ├── Catalog.jsx
│   │   ├── Members.jsx
│   │   ├── Borrow.jsx
│   │   ├── Return.jsx
│   │   └── CreateStaff.jsx
│   ├── portal/           # Member-facing portal
│   │   ├── LandingPage.jsx
│   │   ├── UnifiedLogin.jsx
│   │   ├── MemberDashboard.jsx
│   │   ├── MemberCatalog.jsx
│   │   └── MyBorrows.jsx
│   ├── context/          # React context providers
│   │   ├── AuthContext.jsx
│   │   └── ToastContext.jsx
│   ├── services/
│   │   └── api.js         # Centralized API calls
│   ├── hooks/
│   │   └── useScrollReveal.js # Scroll animation hook

```

10.3 API Endpoints

Method	Endpoint	Description	Access
POST	/api/auth/login	Staff login	Public
POST	/api/auth/register	Create staff account	Protected
GET	/api/auth/setup-status	Check if setup needed	Public
POST	/api/auth/setup	First-run admin setup	Public
POST	/api/auth/staff-register	Staff self-register with invite code	Public
POST	/api/member-auth/register	Member self-registration	Public
POST	/api/member-auth/login	Member login	Public
GET	/api/books	Get all books (with filters)	Public
POST	/api/books	Add a book	Protected
PUT	/api/books/:id	Update a book	Protected
DELETE	/api/books/:id	Delete a book	Protected
GET	/api/members	Get all members	Protected
POST	/api/members	Register a member	Protected
GET	/api/borrows	Get borrow records	Protected
POST	/api/borrows	Issue a book	Protected
PUT	/api/borrows/:id/return	Return a book	Protected
GET	/api/borrows/stats	Dashboard statistics	Protected
GET	/api/portal/my-borrows	Member's own borrows	Member
POST	/api/portal/request-borrow	Member borrow request	Member

11. System Features

11.1 First-Run Setup Wizard

When the system is launched for the first time with no admin account, a setup wizard appears guiding the administrator through creating the first admin account. This eliminates the need for manual database seeding via the command line.

11.2 Role-Based Access Control

The system implements three distinct user roles:

- **Member:** Can browse catalog, borrow books, view personal history
- **Librarian:** Can manage all library operations but cannot create staff accounts
- **Administrator:** Full access including staff account creation

11.3 Unified Authentication

A single login and registration page serves all user types. Users select their role (Member or Staff) before logging in, and the system routes them to the appropriate dashboard automatically.

11.4 Book Catalog Management

The catalog supports adding, editing, and deleting books with full details including title, author, ISBN, category, publisher, publication year, number of copies, and description. The system automatically tracks available copies as books are borrowed and returned.

11.5 Automated Fine Calculation

When a book is returned, the system automatically calculates any overdue fine at \$0.50 per day past the due date. The return confirmation modal shows the exact fine before processing.

11.6 Staff Invite Code System

Staff members can self-register using a secret invite code set by the administrator in the environment configuration. This prevents unauthorized staff account creation while allowing legitimate staff to register independently.

11.7 Email Notifications

When an administrator creates a staff account from the dashboard, the system automatically sends a welcome email to the new staff member containing their login credentials, role, and the staff invite code. This is implemented using Nodemailer with Gmail.

11.8 Real-Time Dashboard Statistics

The administrator dashboard displays live statistics including total books, active members, currently borrowed books, returned books, and overdue books, with automatic status updates when overdue records are detected.

11.9 Scroll-Reveal Animations

The landing page features smooth scroll-triggered animations using a custom IntersectionObserver hook, with content fading in as users scroll down the page.

11.10 Mobile Responsiveness

The entire application is fully responsive with three breakpoints (900px, 768px, 480px), ensuring a consistent experience on smartphones, tablets, and desktops.

12. Technologies Used

Technology	Version	Purpose
React.js	19.2.8	Frontend UI library
Vite	8.2.0	Frontend build tool
Node.js	24.14.1	Backend runtime

Technology	Version	Purpose
Express.js	4.19.2	Backend web framework
MongoDB Atlas	Cloud	NoSQL database
Mongoose	8.4.4	MongoDB object modeling
JSON Web Token	9.0.2	Authentication tokens
bcryptjs	2.4.3	Password hashing
Nodemailer	6.9.14	Email service
Nodemon	3.1.4	Development auto-restart
CORS	2.8.5	Cross-origin resource sharing
dotenv	16.4.5	Environment variable management
Google Material Icons	CDN	UI icon library
Render	Cloud	Backend deployment
Vercel	Cloud	Frontend deployment
Git / GitHub	—	Version control

13. Database Design

13.1 User Schema

```
{
  name:      String (required),
  email:     String (required, unique),
  password:  String (hashed with bcrypt),
  role:      String (enum: ['librarian', 'admin'])
}
```

13.2 Member Schema

```
{
  name:      String (required),
  email:     String (required, unique),
  password:  String (hashed, optional),
  memberId:  String (auto-generated: LIB-0001),
  phone:     String,
  address:   String,
  membershipStatus: String (enum: ['active', 'suspended', 'expired']),
  booksCurrentlyBorrowed: Number (default: 0),
}
```

```
totalBorrowedAllTime:  Number (default: 0)
}
```

13.3 Book Schema

```
{
  title:      String (required),
  author:     String (required),
  isbn:       String (required, unique),
  category:   String (required),
  publisher:   String,
  publishedYear: Number,
  totalCopies: Number (required),
  availableCopies: Number (auto-managed),
  description: String,
  status:     String (auto: 'available'/'unavailable')
}
```

13.4 Borrow Schema

```
{
  book:      ObjectId (ref: Book),
  member:    ObjectId (ref: Member),
  borrowDate: Date (default: now),
  dueDate:   Date (required),
  returnDate: Date (null until returned),
  status:    String (enum: ['borrowed', 'returned', 'overdue']),
  fine:      Number (calculated on return),
  finePaid:  Boolean
}
```

14. System Architecture

14.1 Frontend Architecture

The React frontend follows a component-based architecture with:

- **Context API** for global state management (authentication, toasts)
- **Custom hooks** for scroll-reveal animations
- **Service layer** (`api.js`) centralizing all API calls
- **Role-based rendering** — the app detects user role and renders the appropriate interface

14.2 Backend Architecture

The Express backend follows the MVC pattern:

- **Models** define data structure and validation
- **Controllers** contain business logic
- **Routes** define API endpoints and apply middleware
- **Middleware** handles JWT authentication and role verification

14.3 Security Implementation

- Passwords are hashed using bcryptjs with a salt factor of 10
- JWTs are signed with a secret key and expire after 7 days
- Protected routes require a valid Bearer token in the Authorization header
- The staff invite code prevents unauthorized staff account creation
- The first-run setup endpoint is automatically disabled after the first admin is created
- Environment variables store all sensitive credentials, never committed to version control

15. Testing

15.1 Functional Testing

Test Case	Expected Result	Actual Result	Status
Member registration	Account created, JWT returned	Account created successfully	PASS
Member login	JWT returned with role: member	Redirected to member portal	PASS
Staff login (admin)	JWT returned with role: admin	Redirected to admin dashboard	PASS
Add book	Book saved to database	Book appears in catalog	PASS
Borrow book	Available copies decremented	Copy count updated correctly	PASS
Return book (on time)	Status updated, no fine	Fine = \$0.00	PASS
Return book (overdue 3 days)	Fine = \$1.50	Fine = \$1.50 calculated correctly	PASS
Invalid login credentials	401 error returned	Toast error message shown	PASS
Wrong staff invite code	403 error returned	Registration rejected	PASS
First-run setup (no admin)	Setup wizard shown	Setup page renders correctly	PASS
Setup attempt with existing admin	403 error returned	Endpoint blocked correctly	PASS
Borrow limit exceeded (5 books)	400 error returned	Error toast shown	PASS
Borrow unavailable book	400 error returned	Error toast shown	PASS

15.2 Responsiveness Testing

Device	Screen Width	Result
Desktop	1440px	Full layout, all columns visible
Laptop	1024px	Minor adjustments, fully functional
Tablet	768px	Panels stack, hamburger menu active
Mobile (large)	480px	Cards layout, font sizes reduced
Mobile (small)	375px	Single column, full functionality

16. Results and Discussion

The Library Management System was successfully designed and implemented, meeting all stated objectives. The system provides a comprehensive solution for library management with the following key achievements:

Role-based access control was successfully implemented with three distinct user types. The system correctly routes users to their respective dashboards upon login without any manual configuration.

Automated fine calculation was verified to be accurate — fines are calculated at exactly \$0.50 per day and displayed to librarians before confirming a return.

The member portal provides a complete self-service experience, allowing members to register independently, browse 24 pre-seeded books across 6 categories, borrow books, and track their history.

Email notifications were tested using Gmail App Passwords and confirmed to deliver welcome emails containing credentials and invite codes to newly created staff accounts.

Cloud deployment was successfully completed with the backend hosted on Render and the frontend on Vercel, making the system publicly accessible at any time from any device.

Mobile responsiveness was achieved across all screen sizes through a systematic three-breakpoint CSS approach, with data tables reflowing as cards on small screens.

The system demonstrates that modern web technologies can be used to build effective, production-ready library management solutions without the complexity and cost associated with traditional enterprise library software.

17. Conclusion and Recommendations

17.1 Conclusion

This project successfully designed and implemented a web-based Library Management System that automates the core processes of a library — book cataloging, member management, borrowing, and returning. The system addresses the limitations of manual library management through a modern, secure, and user-friendly platform.

The use of the MERN stack proved to be an effective choice for this project, providing a consistent JavaScript ecosystem across the entire application. MongoDB's flexible document model was well-suited for storing library data, while React's component architecture enabled a clean and maintainable frontend codebase.

The system is fully deployed and accessible in production, demonstrating that a complete, professional-grade library management solution can be built and deployed using entirely free and open-source technologies.

17.2 Recommendations

For future development, the following enhancements are recommended:

1. **Barcode scanning integration** — Allow librarians to scan book barcodes using a webcam or external scanner for faster checkout.
2. **Online fine payment** — Integrate a payment gateway (such as Paystack or Flutterwave) to allow members to pay fines online.
3. **Book reservation system** — Allow members to reserve books that are currently borrowed.
4. **SMS notifications** — Add SMS alerts for overdue books using services like Twilio.
5. **Reports and analytics** — Generate PDF or Excel reports on borrowing trends, popular books, and member activity.
6. **Multi-branch support** — Extend the system to manage multiple library branches under one platform.
7. **Book cover images** — Integrate with a book API (such as Open Library) to automatically fetch and display book cover images.

18. References

1. Afolabi, T. and Adeola, F. (2019). "Automation of Library Services: A Case Study of University Libraries in Nigeria." *Journal of Library and Information Science*, 12(3), pp. 45-58.
2. Okafor, C., Nwosu, I. and Chukwu, A. (2020). "Web-Based Library Information Systems: Design, Development and Implementation." *International Journal of Computer Applications*, 175(12), pp. 1-8.
3. Subramanian, V. (2019). *Pro MERN Stack: Full Stack Web App Development with Mongo, Express, React, and Node* (2nd ed.). Apress.
4. MongoDB Inc. (2024). *MongoDB Atlas Documentation*. Available at: <https://www.mongodb.com/docs/atlas/>
5. Meta Open Source (2024). *React Documentation*. Available at: <https://react.dev/>
6. OpenJS Foundation (2024). *Node.js Documentation*. Available at: <https://nodejs.org/docs/>
7. Express.js (2024). *Express 4.x API Reference*. Available at: <https://expressjs.com/en/api.html>
8. Auth0 (2024). *Introduction to JSON Web Tokens*. Available at: <https://jwt.io/introduction/>
9. Render (2024). *Render Documentation*. Available at: <https://render.com/docs>
10. Vercel (2024). *Vercel Documentation*. Available at: <https://vercel.com/docs>

Word Count: Approximately 3,800 words **Prepared by:** [Your Full Name] **Date:** August 2026